

Omphalos · Advisor presentation script

Spoken text for the 31-slide presentation

13 September 2026

This is a speaking script, written in your voice. **Read the ordinary paragraphs aloud; use the italic directions silently.** The bold phrases are places to put emphasis, not extra words to read.

For the **15-20 minute route**, use slides **1, 3, 4, 9, 10, 11, 13, 14, 15, 16, 19, 20, 21, 22 and 24**. The other main slides expand the discussion; the seven backups have short answers ready if needed. Allow roughly 25 minutes of speaking for the full main deck, with additional time for questions. These are rehearsal estimates, not a requirement to rush.

The script follows the presentation's evidence snapshot: the audited development history through September 12 and the completed September 13 resource-completion cycle. It does not assign results to ongoing work in the other session. No live code visit is necessary.

01 · ACE, control and the cost of a proof

Since our last meeting, I have implemented and evaluated several versions of ACE, but also investigated the controller around it: how we allocate money, expose verifier feedback and decide whether another proof attempt can run.

The three numbers here summarize different parts of that work. We obtained a **55.5% spending reduction**, with three fewer validation solves. In a separate comparison within the current bounded program, adding the Luna book improved coverage by **three problems out of forty**. And Terra reached **38 out of 40 during book creation**, although its later frozen-book evaluation was lower.

I want to explain what those results mean, why they differ, and which contributions are strongest for the thesis. The evaluations discussed here use development data, including a validation partition that we have reused.

Pause briefly. Do not let the three numbers sound like one experiment.

02 · The questions this update answers

There are four questions behind the update. First, whether we have a faithful, inspectable ACE implementation. Second, whether the advice actually increases full-proof coverage. Third, whether using a stronger model as solver or writer changes that conclusion. And finally, what we should retain from the experiments that did not improve performance.

I will keep **implementation, local correctness, coverage and cost** separate. We have made substantial progress on the first two, and found useful control trade-offs. The evidence

for a general ACE coverage gain is still limited.

03 · Our ACE loop, in one slide

You already know ACE, so I will focus on our local implementation. The Generator attempts a Rocq proof using the current playbook. Rocq supplies execution feedback; the Reflector diagnoses the trajectory; and the Curator proposes structured changes to the book.

Our batched implementation also has a **Reducer**, which selects or combines proposals before deterministic code merges and stores the updated book. Delphyne separates those strategies from the policies that choose models, search and resource allocation.

During training, the book evolves between batches. During evaluation, it is **frozen**. We are adapting the context, not updating model weights. That training-versus-evaluation distinction will matter when we discuss 38 out of 40.

04 · Where each ACE element lives in the code

This slide connects the conceptual loop to the actual implementation. The original Generator, Reflector, Curator and Reducer strategies are in `prove_ace.py`, together with their query types. The book representation and deterministic updates live in `ace_playbook.py`; the store records content-addressed versions.

The adaptation driver coordinates those roles and records which book each job used. The more recent bounded proof loop lives separately in `prove_grounded.py`, with verifier operations and campaign budgeting in their own modules.

The useful architectural distinction is that **the model proposes a change, but deterministic code owns the persistent state**. These references also let us inspect the implementation without relying on a verbal description.

Optional visit: open `prove_ace.py` and search `reflect_on_trajectory`, then `aggregate_curation_deltas`. All paths on the slide are relative to `examples/omphalos/`; use symbols rather than line numbers.

05 · What counts as a solve—and what is capped?

Before comparing results, I want to make the measurement rules explicit. A solve needs final verifier acceptance. A qualified solve also satisfies the stated cost rule, and the panel bill includes money spent on failed attempts.

The main X experiments allow **ten cents per problem for Luna** and **one dollar for Terra**. The bounded evaluation uses up to 64 requests and 300 seconds of aggregate verifier time. Terra's training Generator used a nominal one-dollar allowance and 32 requests, with an older stopping rule.

The distinction is that a nominal allowance is not necessarily a hard final bill. Older policies stopped after a crossing request; the bounded controller reserves money before admitting it. That can leave some money unspent.

06 · Development cycles 1/4 · replication and X3

The first replication changed the way the solver worked more than the number of theorems it solved. ACE and the baseline both solved 32 of 40 evaluation cells on the original smaller validation set. Some outputs became shorter, but overall coverage did not improve.

The X experiments added more complete adaptation machinery and larger panels. Offline v2 tied its baseline. Cold online adaptation was slightly better in the recorded runs, while warm starting and additional epochs did not reliably help.

X3 reached **57 of 80 cells against 54 for the baseline**, and its one-epoch book became the later reference. But removing the Reflector also gave 57, so we cannot attribute that entire difference to reflection.

Do not read every row. Explain that 80 cells means forty problems evaluated twice, not eighty independent theorems.

07 · Development cycles 2/4 · better advice and delivery

The next group of experiments tried to make advice more concrete, better grounded and cheaper to deliver. We tested stronger writers, richer failure summaries, different renderings, advice triggered by errors and repair-derived bullets.

There were useful local observations, but **no consistent large coverage improvement over X3**. One particularly useful correction concerned rendering: a cheaper format for one book became more expensive for another. We had to withdraw the general recommendation and describe an interaction instead.

Stall stopping reduced spending while losing a solve, and the simple digest table performed worse than the learned reference. So learned content may contain useful information, but correctness or efficient delivery alone does not guarantee that it finishes more proofs.

08 · Development cycles 3/4 · control and local checks

This is where the clearest budgeting result appears. The bounded money-and-focused configuration cut spending substantially, at the cost of three validation solves. We retained that as an explicit trade-off, while preserving the fact that the original no-loss gate failed.

A follow-up recovered the three solves at higher cost. The polish campaign tied coverage and cost more, so it stayed optional.

We also studied smaller local decisions, such as whether a syntax transformation was valid and useful. Checked transformations improved local correctness, but that is a different endpoint from a completed theorem. The central lesson was to ask **whether a correct local step creates useful progress and whether the search can finish afterward**.

09 · Development cycles 4/4 · coverage and attribution

The later cycles tested effort changes, examples, structured continuation, exploration, syntax repair and compaction. Some pilots looked encouraging, but the full validation comparisons generally did not improve the reference. All four isolated coverage treatments had fewer validation solves and higher spending.

The failures were not all the same. Some structured treatments were confounded by a parser defect. Some new examples never reached the cells where solve outcomes changed. Compaction sometimes created long, repetitive tool-only histories.

That led to the model-and-book attribution study, which I will focus on next. The table also includes the completed September 13 cycle: its smaller response cap and corrected continuation did not win either. **The outcomes are retained as evidence; the losing methods are not automatically promoted.**

On the short route, introduce this as the final part of the history and mention that the preceding three tables provide the full chronology.

10 · The strongest result is a budget trade-off

Here is the accepted budgeting result in concrete terms. Against this campaign's older X3 reference, the bounded configuration reduced the bill from about **\$1.52 to \$0.68** across forty validation problems. Coverage fell from **28 to 25 solves**.

That is a 55.5% spending reduction, and the whole-panel cost per solve almost halves. We explicitly accepted the lower-cost configuration as a useful trade-off. We did not relabel the original gate: a requirement to save money without losing solves was not met.

Both programs had a nominal ten-cent allowance per problem. The difference comes from the controller's behavior under that allowance. For the thesis, this is a concrete result about **resource control**, with an explicit coverage cost.

Point separately to the coverage bars and the spending bars.

11 · A changed baseline explains part of the confusion

An important finding was that we had been discussing several different baselines as though they were interchangeable. The earlier generic program without a book solved 33 training and 27 validation problems. The newer bounded program without a book solved 28 and 24. Adding the Luna book to that newer program gave 28 and 27.

So the no-book program itself lost coverage between these recorded versions. **ACE cannot explain a loss that is already present without ACE.** Within the newer program, the book adds three validation solves. Against the older program, the final ACE configuration matches validation coverage at a lower recorded bill, while losing training coverage.

The older-to-newer comparison is historical and changes several conditions; it does not isolate one controller feature.

12 · Luna's book helps here—but the signal is small

This is the cleanest intended ACE comparison: the same bounded Luna program, with and without the frozen Luna book. Coverage rises from **24 to 27 out of 40**, while the recorded bill falls from about **\$0.78 to \$0.67**.

The three differences all favor the book, but there are only three discordant problems, giving an exact paired p-value of **0.25**. I would describe that as encouraging development evidence, not a statistically supported general gain. It also does not establish that ACE has no value.

The spending result needs another qualification: cache discounts materially affect it. I will show that separately rather than treating a lower bill as proof that the model performed less computation.

13 · The full crossover separates solver from book

We then crossed the solver model with the book source. Each cell contains forty validation problems, with failed attempts included in the bill. Luna uses a ten-cent allowance per problem; Terra uses one dollar, reflecting the recorded tenfold tariff difference.

Without any book, Terra solves **29**, compared with Luna's **24**. Giving both models the same Luna book adds **three solves for Luna and none for Terra**. Terra's own book brings Terra to **30**, but gives Luna no coverage improvement over its no-book result.

The practical cost difference is large: with their own books, Terra costs about **9.67 times as much** for three additional validation solves. This separates the value of the stronger solver from the incremental value of ACE.

Read the matrix by rows first, then compare the same book across rows. Do not add book preparation to these inference figures without saying so.

14 · What the Luna / Terra experiment changes

My interpretation is that the stronger solver has more observed coverage in this setting, but **stronger models do not clearly unlock a larger ACE increment**. A Terra-written book did not improve Luna, and Terra's own increment was only one problem.

That is not proof that stronger writing can never help. The effects are small and uncertain. It does mean that simply scaling the writer is not supported as the main explanation or the obvious next improvement.

The study also separates successful experience from a useful final book. Terra produced more successful training trajectories, yet the retained advice still had to survive selection, preserve valid syntax, reach the right operation and help complete a proof. That chain gives us more specific things to investigate than model scale alone.

15 · Yes: Terra achieved 38/40 during book creation

The 38-out-of-40 result is real, but its setting matters. It comes from **Terra generating proofs on trainX while the book was being built**, starting with an empty book and updating it between four-problem batches.

The Generator spent about **\$6.00 across all forty tasks**. Including reflection, curation, reduction and the recorded embeddings, the full preparation bill was about **\$8.10**. The Generator's bill is the relevant number for its proof attempts; the larger number is the cost of producing the book.

Later, Terra with the finished book solved **32 out of 40 on trainX** under the bounded evaluation controller. Separately, it solved **30 out of 40 on validationX**. We should not present those as a continuous decline on one evaluation: only 38 versus 32 concerns the same training partition.

16 · Why did 38 become 32? What we know—and do not

We know several conditions changed. Training used an evolving book, the older proof loop and a post-crossing stopping rule. Evaluation used a frozen book, focused queries, bounded feedback and conservative reservations before requests.

Although evaluation allowed more requests on paper, **a larger request allowance does not guarantee that more useful calls are admitted**. Verifier limits, parsing and the opportunity to submit the final proof also matter.

What we do not know is how much of the six-solve gap each factor explains. These are different sampled trajectories with multiple changed conditions, not a paired ablation of the controller. I would therefore treat the gap as a reason to investigate lost search opportunity, not as evidence that the finished book destroyed six proofs or that all six losses were caused by admission.

17 · Creating a book is a real up-front expense

The preparation bill is separate from the inference comparisons. Luna's known preparation cost is about **\$0.92**, with its historical embedding charge unavailable. Terra's is about **\$8.10**, including embeddings.

Most of that expense is generating training experience. Both books finish with 26 bullets, and neither reaches the four-thousand-token guard. So a global size limit does not explain all of the information-selection problems we observed.

The economic question is whether useful lessons survive that process and save enough on later attempts to repay preparation. We can calculate conditional break-even workloads, but we cannot assume that the same savings transfer unchanged to a new workload.

18 · Spent versus solved: the actual recorded staircase

These curves include every problem's recorded cost in a fixed order. Moving right means more spending; moving up means another qualified solve. **A flat stretch is money spent without adding a solve.**

For Luna, the own-book endpoint has more solves and a lower bill than no book. For Terra, the own book adds one solve and slightly reduces the bill. The panels have different dollar scales because the actual model prices differ substantially.

The staircase represents accounting over the recorded tasks. It is not an optimized scheduling policy, and it does not show what would happen if we reran the solver at a smaller cap. The endpoints are the safest comparisons to carry away.

Trace one flat segment, then point to the endpoints. Do not narrate every step.

19 · Cache discounts change the economic conclusion

There is a subtle but important limit to the cost claim. At the recorded tariff, Luna's own book saves about **14%**. But if we price exactly the same input tokens without a cache discount, the book becomes about **1.45% more expensive**.

Cached-input share rises from roughly 65% to 76%, while input and output token counts both rise slightly. Terra's own-book uncached sensitivity is also an increase, about 8.5%.

The observed bill saving is real and practically useful. What changes is the explanation: **we cannot attribute all of it to less reasoning or fewer tokens**. Its economics depend partly on context reuse and the serving tariff. This calculation changes prices on recorded trajectories; it does not simulate new executions.

20 · Four failure mechanisms behind weak transfer

Four examples explain why a plausible lesson may fail to improve coverage. First, in one successful training case, the accepted proof ends with `nia`, but the last proposed script does not show that ending. The learning process needs the exact accepted evidence, including assistance.

Second, the Reducer turned valid `assert` syntax into rejected `have` syntax around an existing lemma. Checking that a name exists would not detect that error.

Third, a parser concatenated multiple final messages, producing 51 structured parse failures. Some intended mechanisms therefore failed before useful delivery. Finally, compaction shortened context but sometimes created repeated tool calls and no proof submission.

The common lesson is that **capture, validity, retention, exposure, use and completion all matter**. Improving one link does not establish an end-to-end gain.

21 · More allowance recovered Terra proofs, then stalled

To examine remaining capacity, we continued already-paid proof paths on selected eight-problem training panels. We did not restart them and count the original work again.

Doubling the money allowance added no Luna solves. Terra improved from **four to six without ACE**, and from **three to seven with the corrected own-book context**. Increasing request and verifier allowances afterward added no further solves.

That final plateau is still not an intrinsic capability ceiling. Eleven residual paths faced monetary admission denial, and one carried an API failure. The experiments show that extra money can recover some Terra proofs, but they also show why **remaining failures cannot simply be labeled problems the model is incapable of solving**.

22 · 13 September: the first completion fixes did not win

The first completed follow-up tested two isolated changes. Arm A reduced the response cap from 32,768 to 4,096 tokens, reducing the amount that had to be reserved for output. Arm B used the corrected structured-continuation interface.

A improved training coverage from 28 to 30, but validation fell to **26 versus the reference's 27**, and cost substantially more. B reached **23 validation solves** and also cost more than the reference.

Neither passed the practical gate, so no follow-up was selected and neither became the default. The completed cycle spent **\$3.61 of the \$30 ceiling**. This is a useful negative result about these particular implementations; it does not settle every possible completion policy. Ongoing work in the other session is not scored in this update.

23 · Are failed ideas inherited by later experiments?

I also checked whether repeatedly committing experiments meant that later runs were accumulating all the unsuccessful methods. In the recent cycles, the main answer is no: the candidate implementations remain available, but methods such as compaction, exploration, repair and continuation are **explicitly opt-in**.

What later experiments do inherit is the accepted bounded controller and its cost-versus-coverage compromise. They also share implementation code, so changing a parser or runtime component can affect several configurations even when experimental switches remain off.

Keeping negative results and reproducible candidates is valuable. But we should distinguish **archiving an experiment, fixing correctness and promoting a new reference**. An explicit reference constructor and checks of shared behavior would make that distinction harder to lose across sessions.

24 · The thesis claim and the advisor decisions

My strongest current thesis claim is that we have implemented ACE within Delphyne, demonstrated useful control over the cost of proof search, and identified concrete ways in which evidence and execution can prevent advice from becoming a completed proof.

I would not yet claim a general, reproducible ACE coverage improvement, an intrinsic Luna ceiling, or a general advantage from stronger writers. Those remain open.

The next improvements should earn their cost by preserving faithful, executable evidence and converting useful local progress into accepted proofs under the full resource budget. For the discussion, I would like your view on **which cost-versus-coverage trade-off best serves the thesis, and what additional evidence is necessary to support that framing**. Development remains on trainX and validationX; testX stays closed.

Stop here and invite discussion. Use the backups to answer the question that is actually asked.

Backup answers

25 · What are the exact crossover costs and caps?

These are the six exact inference totals. Each row covers forty validation problems, including unsuccessful attempts. Luna's cap is ten cents per problem and Terra's is one dollar; preparation is excluded.

The main comparison I would emphasize is Luna's own book: **27 solves for about \$0.67**, against 24 for about \$0.78 without the book. Terra's own book reaches **30 for about \$6.45**, against 29 for about \$6.61 without it. The reused observations do not turn this into 240 independent theorem samples.

26 · Which historical books improved coverage?

X3 and the no-Reflector variant both reach **57 of 80 cells**, against 54 for no ACE. That is a modest descriptive difference, and it does not isolate reflection as the cause. Several more elaborate variants score lower.

The repair-bullet variant has 56 raw successes, but only 55 meet the retrospective cost qualification. I have kept that distinction rather than silently changing the archived raw result. These arms share repeatedly used development problems and differ in their books or delivery methods; they are not independent confirmations of one effect.

27 · How can a run stop while money remains?

The controller reserves an upper bound before making a request. That includes an input estimate and the maximum permitted output charge. In this recorded example, about **\$0.085742 remains**, but the next request requires **\$0.086368** reserved, so it is denied.

The actual hypothetical bill might have been smaller, but it is unknown because the request did not run. Lowering the output cap can reduce the reservation, while also restricting generation and reasoning. Our first such trial did not improve validation performance, so the problem needs more care than simply making admission less conservative.

Optional visit: `runtime/campaign_budget.py`, then `CampaignResponsesModel.estimate_budget` and `Ledger.reserve`.

28 · When would preparing the book pay for itself?

If the observed per-attempt inference savings continued unchanged, Luna's known preparation cost would break even after roughly **336 attempted problems**, and Terra's after roughly **2,034**.

Those are arithmetic scenarios, not predictions. They assume the same task mix, usefulness, prices and cache reuse. They include unsuccessful attempts in the workload denominator. Luna also has a missing historical embedding charge, and neither figure includes engineering or assistant costs. I would use these numbers to explain the importance of preparation, not to promise a deployment return.

29 · Which evidence should an ACE lesson retain?

At minimum, I want the proposed script, the exact execution and accepted terminal proof, and the context in which any retained snippet was checked. The successful ending must include automatic assistance when that is what actually finished the proof.

The `amc12_2000_p6` witness illustrates the issue: the proof succeeds with `nia`, although the ending is absent from the last proposed text. A missing-text flag does not mean the proof failed. After preserving the source correctly, we still need evidence that a later operation received and used the lesson, and that this helped complete a theorem economically.

Optional visit: `report/ace_retrospective/data/creator_terminal_evidence.json`, entry 0.

30 · Why is +3 solves not yet a supported gain?

There are three gains and no losses, but only three discordant problems. The exact paired p-value is **0.25**, above our current support threshold of 0.10. That limits a statistical claim; it does not establish zero benefit.

We also need to keep repeated seeds and related theorem families together, preserve failed cells and costs, and recognize that `validationX` has been reused for development. A promising pilot can still justify a bounded follow-up. That decision is separate from statistical support or promotion, and it should not involve buying extra runs merely to cross a significance threshold.

31 · Which files should I rehearse before presenting?

These are the optional places to inspect the claims directly. `prove_ace.py` shows the strategy mapping; the `playbook` and `store` modules show deterministic state management; the attribution report contains the 38-out-of-40 preparation result; and the budgeting module explains request admission.

The terminal-evidence export provides the concrete accepted-proof witness. I would navigate by function or class name, because active implementation work can move line numbers. The slides already contain the explanations, so these visits are for inspection rather than filling in missing content, and none requires launching an experiment.